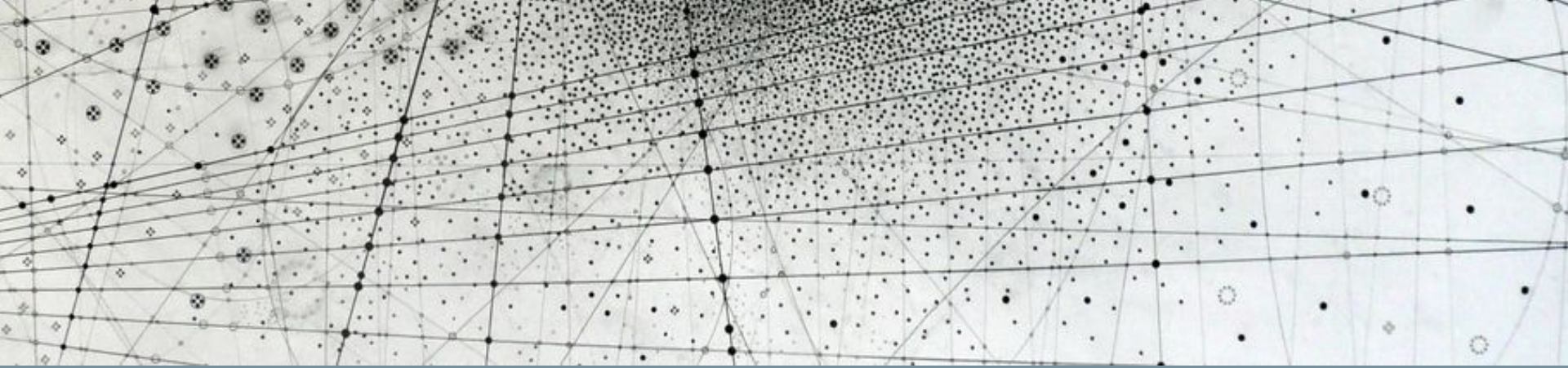




Research Methods in Machine Learning & Physics

Announcements

Due in 1 week:

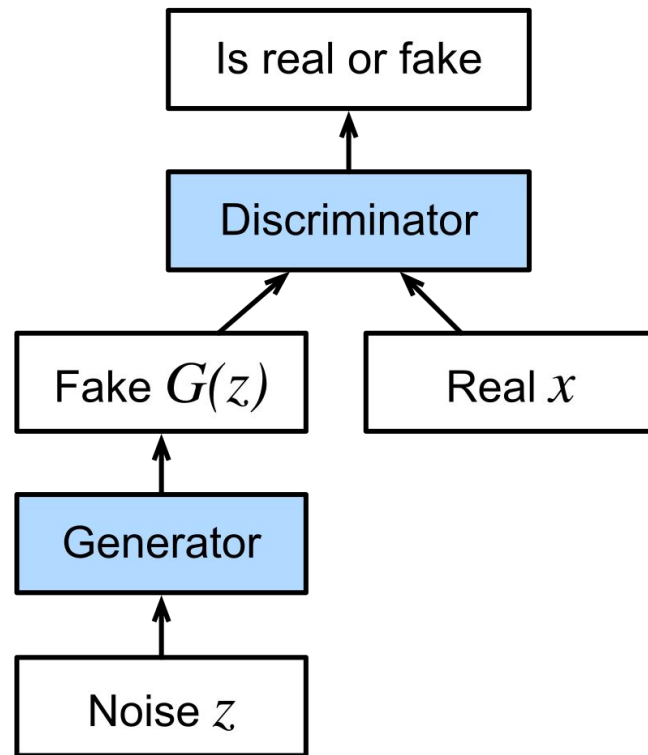
Pset 2 (GNNs applied to particle physics data)

Anonymous peer-review of Project 1

Quick recap

Quick Recap: GANs

- **Generative Adversarial Networks** (GANs) pit two models against each other:
 - A **generator** of outputs (i.e. the "artist")
 - A **discriminator** (i.e. the "art critic")
- The generator tries to generate samples that are indistinguishable from real samples
- The discriminator tries to effectively tell real samples apart from generated ones
- Optimizing both sets of model parameters results in a generator that can make realistic samples

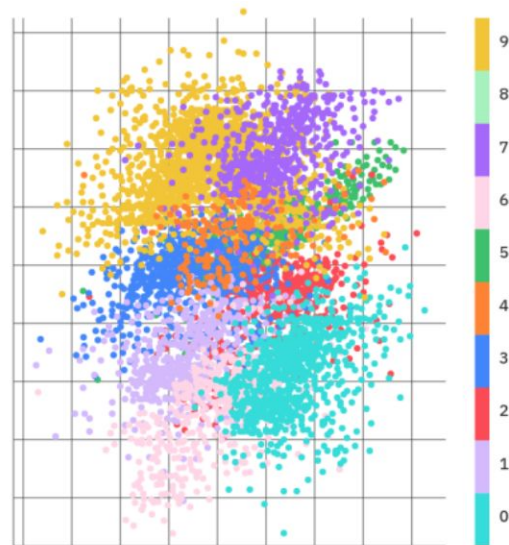
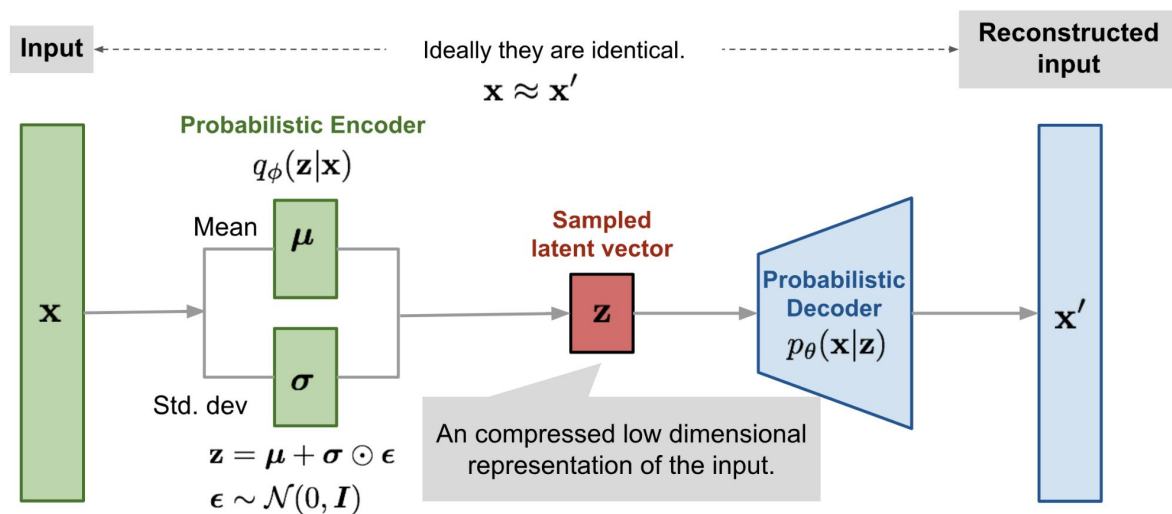


Quick Recap: VAEs

Variational autoencoders learn 2 values for every latent dimension: μ and σ .

These values specify a Gaussian distribution for each latent dimension.

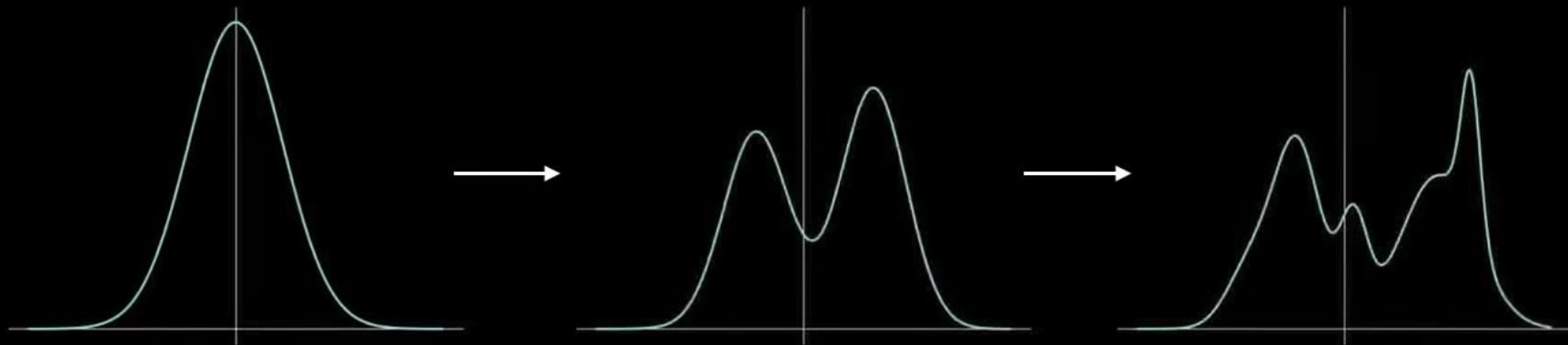
Sampling from these Gaussians, we find our latent variables z in a more ordered latent space.



Generative models, Part II: Flow-based methods

“Flow-based methods” or “transport methods”:

Learning how to turn a simple distribution into a complex one



How these models improve on GANs & VAEs

GANs & VAEs, as we saw last class, don't actually learn the exact likelihood distribution of our data.

Sometimes that's sufficient for our needs – maybe if we're generating cat photos, we don't need to understand the underlying distribution of cats – but in physics, we *do* often care about this.

Plus, those previous models have some limitations:

- **GANs** are trained using adversarial models, which is an inherently unstable and finicky process
- **VAEs** are trained using a variational lower bound (ELBO), rather than the exact likelihood.
 - They can also tend to produce blurry samples due to the MSE loss averaging over uncertainties

Normalizing Flows

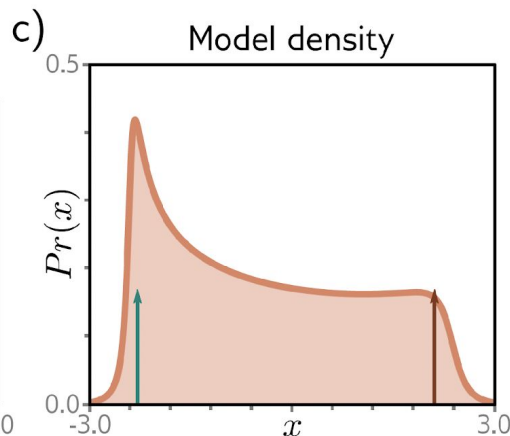
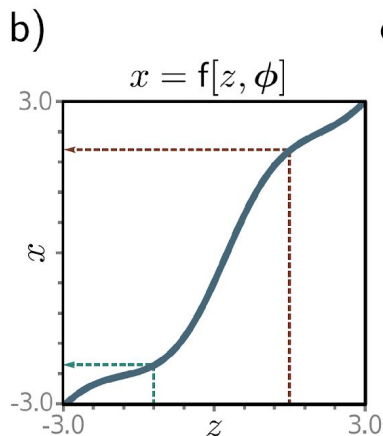
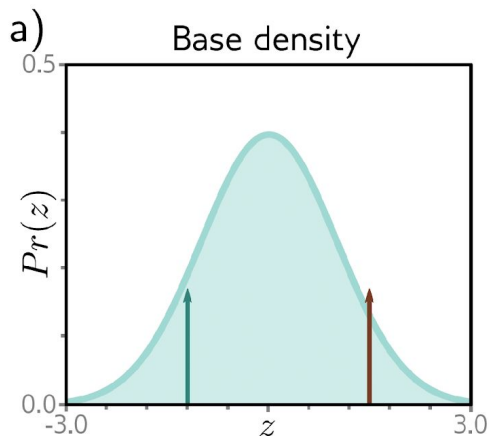
Normalizing flows (NFs)

Normalizing Flows (NFs) learn an **invertible transformation** between a simple base distribution (e.g. a Gaussian) and the data distribution (i.e. complex, unknown).

How it works:

1. Sample z from a simple Gaussian distribution
2. Learn a series of invertible transformations f that take z to x (i.e. your data)

$$\mathbf{x} = \mathbf{f}[\mathbf{z}, \boldsymbol{\phi}] = \mathbf{f}_K \left[\mathbf{f}_{K-1} \left[\dots \mathbf{f}_2 \left[\mathbf{f}_1[\mathbf{z}, \boldsymbol{\phi}_1], \boldsymbol{\phi}_2 \right], \dots \boldsymbol{\phi}_{K-1} \right], \boldsymbol{\phi}_K \right]$$



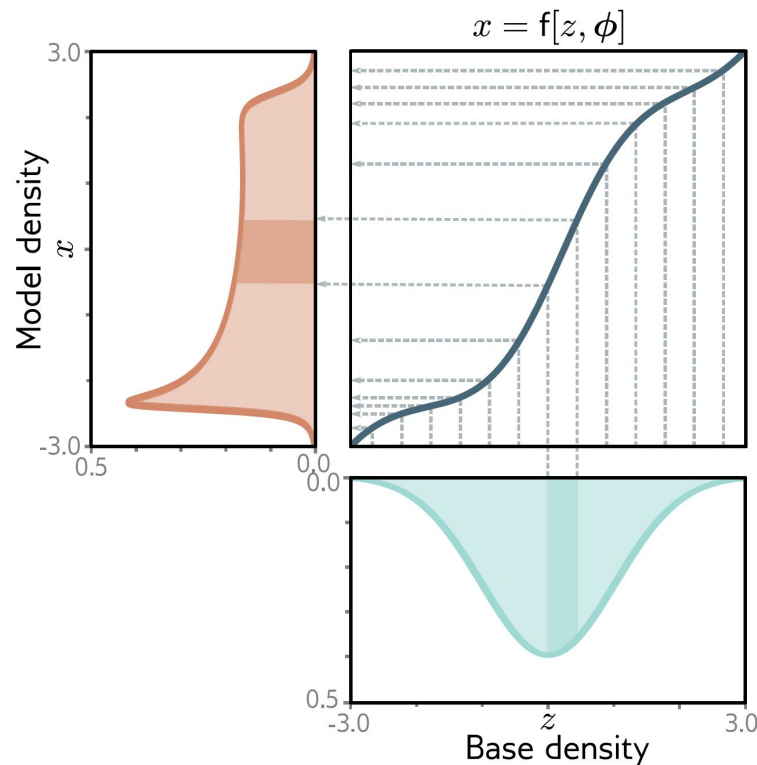
Normalizing flows (NFs)

Normalizing Flows (NFs) learn an **invertible transformation** between a simple base distribution (e.g. a Gaussian) and the data distribution (i.e. complex, unknown).

They use the change of variables formula to compute the exact likelihoods at each step, **ensuring the total probability is maintained**:

$$Pr(x|\phi) = \left| \frac{\partial f[z, \phi]}{\partial z} \right|^{-1} \cdot Pr(z)$$

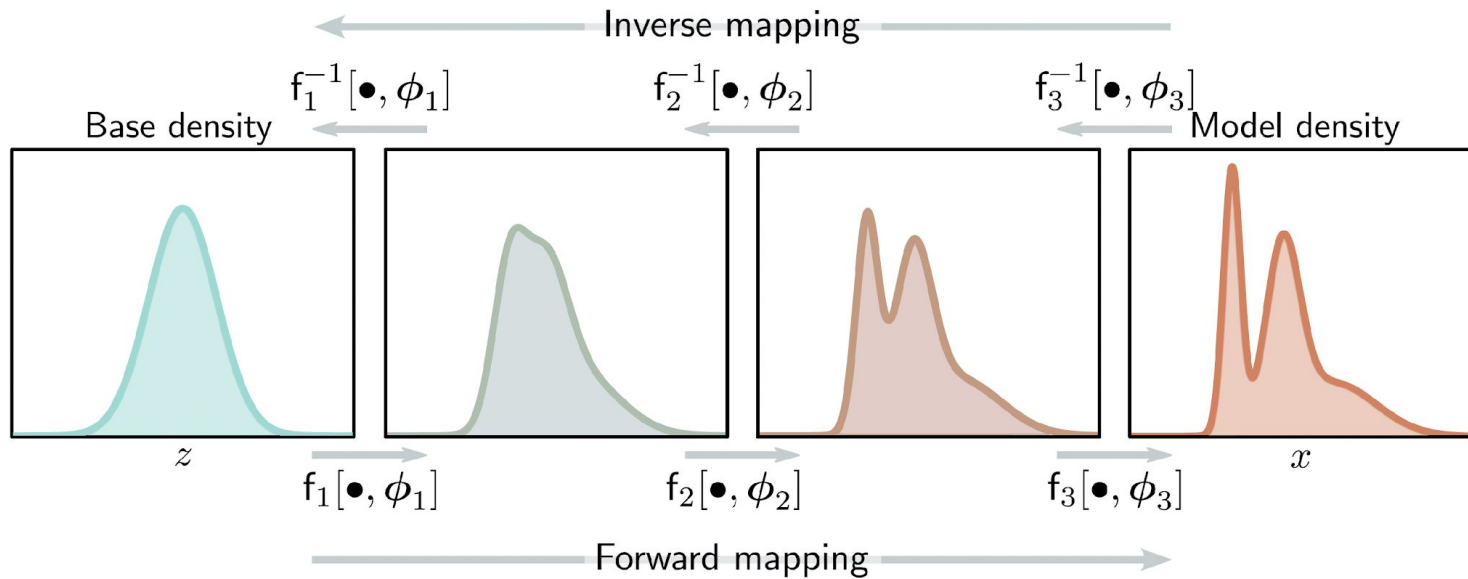
Inverse determinant
of the Jacobian



Normalizing flows (NFs)

They are called “Normalizing Flows” because, in practice, **they learn the inverse mapping**, i.e. how to turn your data distribution into the base density (the normal distribution).

$$\mathbf{z} = \mathbf{f}^{-1}[\mathbf{x}, \phi] = \mathbf{f}_1^{-1} \left[\mathbf{f}_2^{-1} \left[\dots \mathbf{f}_{K-1}^{-1} \left[\mathbf{f}_K^{-1}[\mathbf{x}, \phi_K], \phi_{K-1} \right], \dots \phi_2 \right], \phi_1 \right]$$



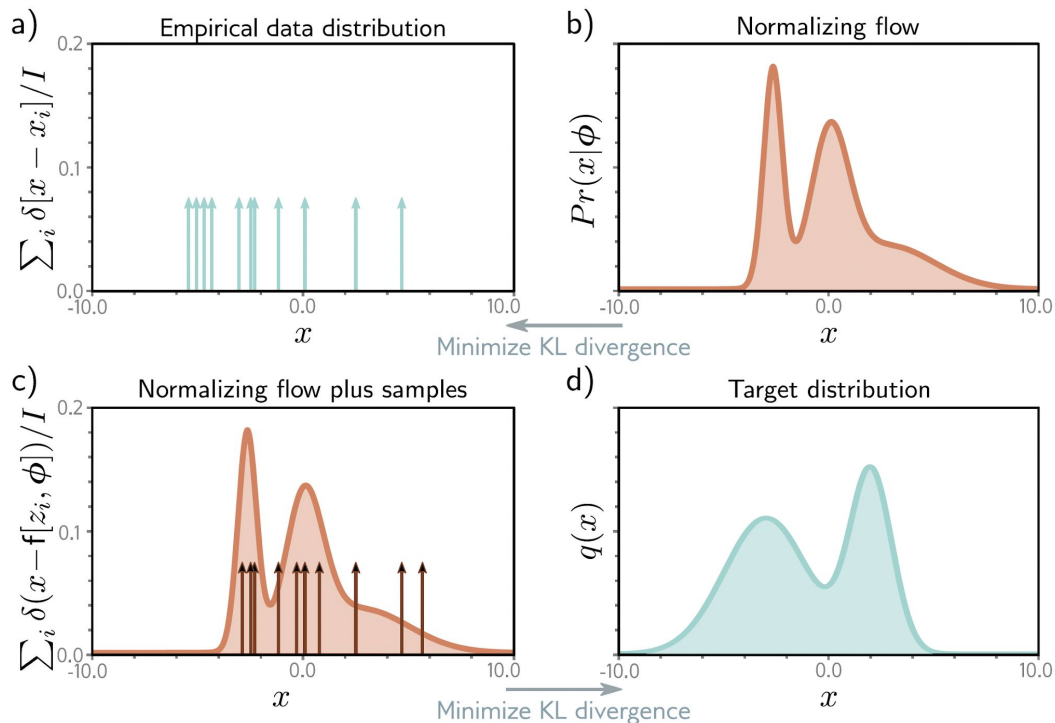
Normalizing flows (NFs)

Each loss function is formulated to maximize the likelihood of the data, i.e. **minimize the negative log-likelihood**:

$$\begin{aligned}\hat{\phi} &= \operatorname{argmax}_{\phi} \left[\prod_{i=1}^I Pr(x_i|\phi) \right] \\ &= \operatorname{argmin}_{\phi} \left[\sum_{i=1}^I -\log \left[Pr(x_i|\phi) \right] \right] \\ &= \operatorname{argmin}_{\phi} \left[\sum_{i=1}^I \log \left[\left| \frac{\partial f[z_i, \phi]}{\partial z_i} \right| \right] - \log [Pr(z_i)] \right]\end{aligned}$$

Normalizing flows (NFs)

These can be used to directly estimate the probability density of a target distribution:



Diffusion Models

Diffusion Models

How to create a sandcastle:

Step 1: Take a sandcastle

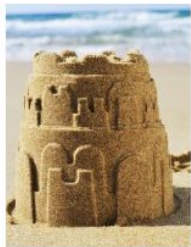
Step 2: Destroy the sandcastle

Step 3: Remember how you destroyed the sandcastle

Step 4: Reverse the process

Key Idea

Once you know how to reconstruct sandcastles, you can start with some different “sand”, apply this process, and end up with a different “sandcastle”



Diffusion Models

Diffusion models are easy-to-train and produce high quality samples (even better than GANs or VAEs).

They are trained to understand how to gradually add & remove random noise from the input data:

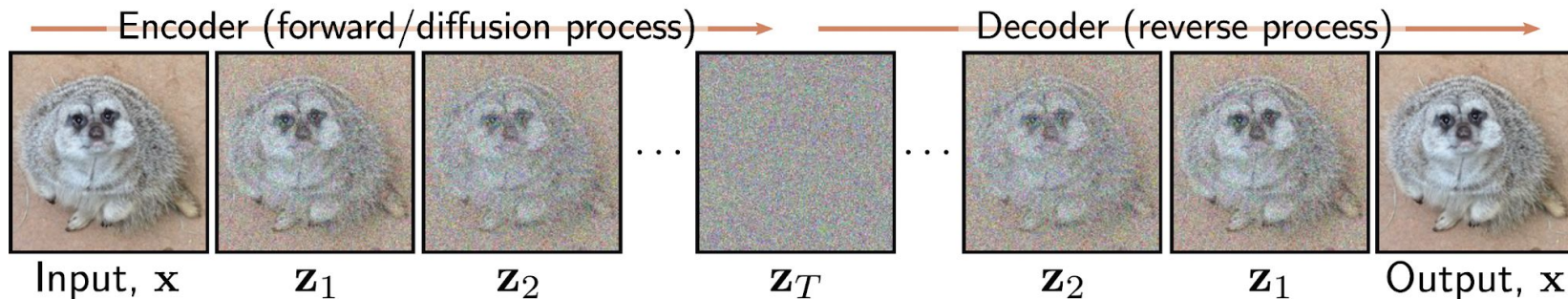


Figure 18.1 Diffusion models. The encoder (forward, or diffusion process) maps the input x through a series of latent variables $z_1 \dots z_T$. This process is pre-specified and gradually mixes the data with noise until only noise remains. The decoder (reverse process) is learned and passes the data back through the latent variables, removing noise at each stage. After training, new examples are generated by sampling noise vectors z_T and passing them through the decoder.

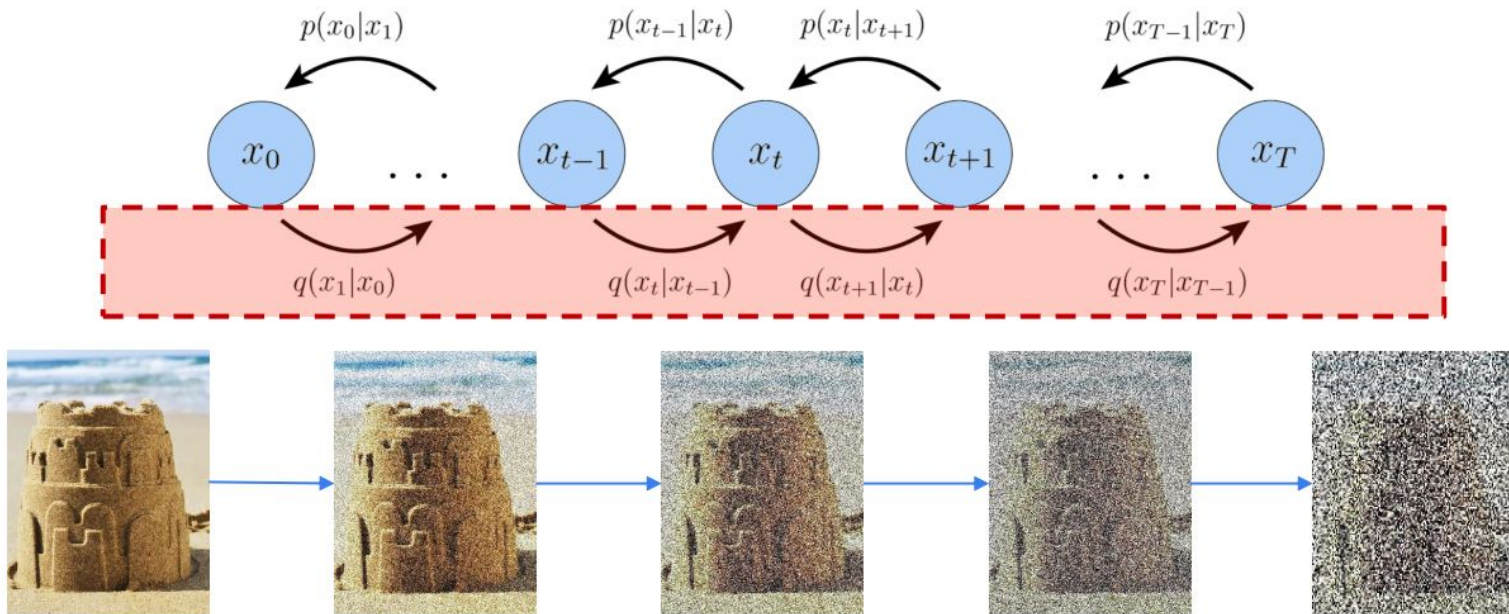
Diffusion Models

Diffusion models are autoencoders

JANUARY 31, 2022

© Reading time ~14 minutes

We gradually add more and more noise until the input is pure noise:



Diffusion Models

Denoising Diffusion Probabilistic Models

Jonathan Ho
UC Berkeley
jonathanho@berkeley.edu

Ajay Jain
UC Berkeley
ajayj@berkeley.edu

Pieter Abbeel
UC Berkeley
pabbeel@cs.berkeley.edu

Algorithm 1 Training

```
1: repeat  
2:    $\mathbf{x}_0 \sim q(\mathbf{x}_0)$   
3:    $t \sim \text{Uniform}(\{1, \dots, T\})$   
4:    $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$   
5:   Take gradient descent step on  
        $\nabla_{\theta} \|\epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, t)\|^2$   
6: until converged
```

Algorithm 2 Sampling

```
1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$   
2: for  $t = T, \dots, 1$  do  
3:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  if  $t > 1$ , else  $\mathbf{z} = \mathbf{0}$   
4:    $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$   
5: end for  
6: return  $\mathbf{x}_0$ 
```

Flow-Matching

X_0



X_1



Flow-Matching

These models lie somewhere in between Normalizing Flows & Diffusion Models!

They bypass some of the restrictions of Normalizing Flows by removing the requirement to calculate the Jacobians & directly regressing a vector field instead of the likelihood-based training objective.

Diffusion Meets Flow Matching: Two Sides of the Same Coin

Flow matching and diffusion models are two popular frameworks in generative modeling. Despite seeming similar, there is some confusion in the community about their exact connection. In this post, we aim to clear up this confusion and show that *diffusion models and Gaussian flow matching are the same*, although different model specifications can lead to different network outputs and sampling schedules. This is great news, it means you can use the two frameworks interchangeably.

AUTHORS

Ruiqi Gao
Emiel Hoogeboom
Jonathan Heek
Valentin De Bortoli
Kevin P. Murphy
Tim Salimans

AFFILIATIONS

Google DeepMind

PUBLISHED

Dec. 2, 2024

Flow-Matching

FLOW MATCHING FOR GENERATIVE MODELING

Yaron Lipman^{1,2} **Ricky T. Q. Chen**¹ **Heli Ben-Hamu**² **Maximilian Nickel**¹ **Matt Le**¹

¹Meta AI (FAIR) ²Weizmann Institute of Science

These models lie somewhere in between Normalizing Flows & Diffusion Models!

Feature	Normalizing Flows	Diffusion Models	
Log-likelihood	Exact (if Jacobian is tractable)	Approximate (ELBO or score matching)	
Jacobian determinant	Required (can be expensive)	Not needed	
Sampling speed	Fast (one pass)	Slow (many steps)	
Invertibility	Mandatory	Not required	
Expressivity	Limited by tractable Jacobians	High	

In-class work

Understanding Deep Learning:

Notebooks 16.1 (1D Normalizing Flow) and 18.2 (1D Diffusion Model)